Member 3: UPI App, Ops Dashboard and Testing

Work assignment for the third student in the 3-member team

UPI Payment Failure Diagnosis, Reconciliation and Fraud Risk Monitoring System

Your job in one line: build everything a person looks at, and prove the whole system works together. Two screens for two audiences: a customer's payment app, and an operations dashboard. Then test every scenario end to end. Your screens must explain, not just display.

1. Your Role and How It Fits

Member	Track	Builds
1	backend/	Database, payment APIs, state machine, idempotency, bank simulators
2	fraud-intel/	Six threaded fraud checks, fraud model, diagnosis decision table, recovery-time estimate
3 (you)	frontend/	UPI app screens, operations dashboard, alerts, end-to-end testing

You only talk to the backend through its HTTP API. Everything you show comes from Member 1's endpoints, which carry Member 2's results. You never decide what a payment state means on the screen: the backend decides, you display. You are also the team's tester, so you are the first to find where the three tracks do not fit.

2. What You Deliver

- **UPI app** (plain HTML, CSS and JavaScript): send money, live payment status, payment history, and a clear banner when a payment is stuck.
- **Operations dashboard** (Streamlit): overview with counts, reconciliation queue, fraud cases, alerts, transaction detail, and demo controls that switch the simulated banks.
- **Alerts** to sender, receiver and operations: in the app, on the dashboard, and optionally by email.
- **A mock backend** so you can build before Member 1 is ready.
- **End-to-end tests** for the 10 scenarios, with results and screenshots in docs/test-scenarios.md.
- **The live demo script** the team uses in front of the faculty.

3. Setup

```
pip install streamlit pandas requests pytest fastapi uvicorn httpx
```

Suggested layout inside frontend/:

```
frontend/
  mock_api.py              fake backend that follows the contract (use until the real one is ready)
  upi-app/
    index.html  style.css  config.js  logic.js  app.js
  ops-dashboard/
    app.py                   Streamlit dashboard
    alert_mailer.py          optional email alerts
  tests/
    test_e2e.py              the 10 scenarios against the API
```

How to run each part:

```
uvicorn mock_api:app --port 8000          # mock backend (or the real backend on the same port)
cd upi-app && python -m http.server 5500    # then open http://localhost:5500
cd ops-dashboard && streamlit run app.py    # opens the dashboard
pytest tests/ -v                            # end-to-end scenarios
```

Serve the app with **python -m http.server** rather than opening the file directly, and ask Member 1 to enable CORS in the backend, otherwise the browser will block your requests.

4. Suggested Timeline

When	Phase	Your tasks	Done when
Days 1-2	0 Design lock	Send Member 1 the list of extra endpoints and fields you need (Step 0)	Both agreed, contract updated
Week 1	1 Skeleton	Steps 1-3 on the mock backend: app screens and dashboard layout	Both screens work with mock data
Week 2	2 Happy path	Point both screens at the real backend	A payment from the app gets a real result
Week 3	3 Core	Stuck-payment banners, reconciliation queue, detail page, alerts, demo controls	A stuck payment is explained on both screens
Week 4	4 Hardening	Step 5: run all 10 scenarios, file issues, retest	All 10 pass with screenshots
Week 5	5 Polish	Demo script rehearsal, report screenshots	Guide's dry-run passes

5. Step-by-Step Tasks

Step 0. Design lock: what you need from the others

The agreed contract covers sending a payment and reading its status, but your screens need a few more things. Ask for these in Phase 0, in writing:

You need	From	Why
GET /transactions returning a list (filter by account_id and state), each with id, created_at, sender, receiver, amount, state, decision, score	Member 1	Payment history in the app and every table in the dashboard. The current contract has no list endpoint.
GET /accounts/{id} returning the balance	Member 1	Show the balance after each payment.
diagnosis and **recovery** fields (optional) inside GET /payment-status/{id}	Members 1 and 2	Show the reason, severity, user message and recovery estimate for stuck payments.
CORS enabled in the backend	Member 1	Otherwise the browser blocks the app from calling the API.
A reliable way to trigger BLOCK in tests (for example a fixed risky amount)	Members 1 and 2	Scenario 10 needs a repeatable fraud block.
Seed account ids and one low-balance account	Member 1	Scenarios 1 and 7.

> **Never code against a field you only assumed. Put every field you use in docs/architecture.md, and treat a change to it as a contract change that needs all three members' agreement.**

Step 1. Use the mock backend

mock_api.py is a small fake backend that follows the contract, so your screens work from day one. It has the same endpoints as the real one plus the extra ones in Step 0. When the real backend is ready you only change the base address.

Endpoint	What the mock does
POST /transactions	Creates a payment. Same key twice returns the same payment with duplicate = true. Rejects bad input with 422.
GET /transactions	Lists payments, newest first, optionally for one account or state.
GET /payment-status/{id}	State, timeline, and for stuck payments a diagnosis and a recovery message.
GET /risk/{id}, /alerts, /cases, /logs/{id}	Fraud breakdown, alerts, fraud cases, event log.
POST /sim/config	Switches the simulated banks: SUCCESS, ERROR, TIMEOUT, LOST_ACK.

In the mock, a payment of 50,000 or more is blocked, 20,000 or more goes to review, and a receiver set to TIMEOUT or ERROR leaves the payment under reconciliation. These are only for building your screens.

Step 2. The UPI app

Screen	What it shows	Calls
Send money	Receiver, amount, Pay button; then a status card with title, message, banner and progress list	POST /transactions, GET /payment-status/{id}
History	Past payments with who, how much and the outcome; tap one to see its status	GET /transactions?account_id=
Header	Balance after every payment	GET /accounts/{id}

Keep logic you want to test in **logic.js**, with no page access, so it can be tested without a browser:

```
// Pure helper functions: no page access, so they are easy to test with Node.
const FINAL_STATES = ["COMPLETED", "FAILED", "RECONCILIATION_REQUIRED"];
const STUCK_STATES = ["TIMEOUT", "RECONCILIATION_REQUIRED"];

function isFinal(state) {
  return FINAL_STATES.includes(state);
}

function isStuck(state) {
  return STUCK_STATES.includes(state);
}

// The screen decides how a state LOOKS. What a state MEANS comes from the backend.
function describeState(state) {
  const map = {
    COMPLETED: { title: "Payment completed", tone: "ok" },
    FAILED: { title: "Payment failed", tone: "bad" },
    RECONCILIATION_REQUIRED: { title: "Payment under review", tone: "warn" },
    TIMEOUT: { title: "Checking your payment", tone: "warn" },
  };
  return map[state] || { title: "Payment in progress", tone: "info" };
}

function makeKey() {
  if (typeof crypto !== "undefined" && crypto.randomUUID) return crypto.randomUUID();
  return "key-" + Date.now() + "-" + Math.random().toString(16).slice(2);
}

function formatMoney(n) {
  return "Rs. " + Number(n).toLocaleString("en-IN", { maximumFractionDigits: 2 });
}

function validateAmount(text) {
  const n = Number(text);
  if (!text || Number.isNaN(n) || n <= 0) return "Enter an amount above zero";
  return null;
}
```

```
if (typeof module !== "undefined") {
  module.exports = { isFinal, isStuck, describeState, makeKey, formatMoney, validateAmount };
}
```

The payment handler is the most important code in the app. Three details matter:

- **One idempotency key per payment attempt.** It is created once and sent with the request.
- **On a network error, keep the same key.** If the request did reach the server, paying again cannot create a second debit. After success, create a new key for the next payment.
- **Disable the Pay button while waiting**, so a double tap cannot send two requests.

```
$("pay-form").addEventListener("submit", async (e) => {
  e.preventDefault();
  const problem = validateAmount($("amount").value);
  if (problem) return showFormError(problem);
  $("form-error").hidden = true;
  $("pay-btn").disabled = true;                // stops double taps
  try {
    const res = await apiCall("POST", "/transactions", {
      idempotency_key: currentKey,
      sender_account_id: CONFIG.MY_ACCOUNT,
      receiver_account_id: $("receiver").value,
      amount: Number($("amount").value),
    });
    await showStatus(res.transaction_id);
    currentKey = makeKey();                    // it reached the server: next payment is new
    $("amount").value = "";
    loadBalance();
    loadHistory();
  } catch (err) {
    // Keep the SAME key. If the request did reach the server, paying again is safe.
    showFormError(err.status ? err.message
      : "Could not reach the server. Tap Pay again. It is safe to retry.");
  } finally {
    $("pay-btn").disabled = false;
  }
});

$("tab-send").addEventListener("click", () => showTab("send"));
$("tab-history").addEventListener("click", () => showTab("history"));
```

The status card turns the backend's answer into something a person understands. Stuck payments show the backend's own user message plus the recovery estimate in a banner. Use **textContent** rather than innerHTML when putting server text on the page, so nothing from the server can be run as code.

```
function renderStatus(s) {
  const d = describeState(s.state);
  $("status-card").hidden = false;
  $("status-title").textContent = d.title;
  $("status-title").className = d.tone;

  const last = s.timeline.length ? s.timeline[s.timeline.length - 1] : null;
  let message = "";
  if (s.state === "COMPLETED") message = "The money has reached the receiver.";
  else if (s.state === "FAILED") message = "Reason: " + ((last && last.detail) || "not available");
  else if (s.diagnosis) message = s.diagnosis.reason;
  $("status-message").textContent = message;

  const banner = $("status-banner");
  if (s.diagnosis) {
    let text = s.diagnosis.user_message;
    if (s.recovery) text += " " + s.recovery.message;
    banner.textContent = text;
    banner.className = "banner" + (s.diagnosis.severity === "HIGH" ? " severe" : "");
    banner.hidden = false;
  } else {
    banner.hidden = true;
  }

  const tl = $("timeline");
  tl.replaceChildren();
```

```
  for (const ev of s.timeline) {
    const li = document.createElement("li");
    li.textContent = ev.to + (ev.detail ? " (" + ev.detail + ")" : "");
    tl.appendChild(li);
  }
}
```

Rules for the status card: COMPLETED says the money reached the receiver; a stuck payment (TIMEOUT or RECONCILIATION_REQUIRED) shows the banner with a clear **do not retry** message; FAILED shows the reason from the timeline; any unknown state shows a neutral "in progress" message rather than an error.

What was tested on the reference code:

Test	Result
Pure logic with Node (final states, stuck states, labels, amount validation, unique keys)	All assertions passed
Page in a simulated browser against the mock: balance shown, empty amount rejected, normal payment completes, banner hidden	Passed
Receiver set to TIMEOUT: status card says under review, banner shows "Do not retry" and the recovery estimate	Passed
History lists both payments	Passed
Server unreachable: message says it is safe to retry, and the same idempotency key is kept	Passed

Step 3. The operations dashboard

The audience is an operations person deciding what to do about a payment. Every flagged row needs a visible reason, not just a label.

Page	Shows	Source
Overview	Counts: total, completed, failed, needing reconciliation, review or blocked; table of all payments	GET /transactions
Reconciliation queue	Payments in RECONCILIATION_REQUIRED or TIMEOUT; open one for the full explanation	GET /transactions, /payment-status
Fraud cases	Cases table; open a payment to see each check's score and reason with a chart	GET /cases, /risk
Alerts	All alerts, filterable by sender, receiver, operations	GET /alerts
Transaction detail	Timeline, diagnosis reason and severity, recovery estimate, fraud breakdown, alerts for this payment	GET /payment-status, /risk, /alerts
Demo controls	Switch the simulated banks for the next payment	POST /sim/config

One helper calls the backend and shows a clear message if it is down, so the dashboard never just crashes. One function, **explain()**, builds the explanation and is reused on three pages:

```
def call(method, path, **kw):
    """Call the backend. Show a clear message instead of crashing if it is down."""
    try:
        r = requests.request(method, API + path, timeout=5, **kw)
        r.raise_for_status()
        return r.json()
    except Exception as e:
        st.error(f"Backend problem on {path}: {e}")
        return None
```

```
def explain(tid):
    """Why was this payment flagged? Risk breakdown, diagnosis, timeline, alerts."""
    status = call("GET", f"/payment-status/{tid}")
    risk = call("GET", f"/risk/{tid}")
    if status:
        st.subheader(f"State: {status['state']}")
        diag = status.get("diagnosis")
        if diag:
            st.warning(f"**Why:** {diag['reason']}  |  **Severity:** {diag['severity']}  |  "
                       f"**Action:** {diag['action']}")
        if status.get("recovery"):
            st.info(status["recovery"]["message"])
        st.write("Payment timeline")
        st.dataframe(pd.DataFrame(status["timeline"]))
    if risk:
        st.write(f"Fraud decision: **{risk['decision']}** (score {risk['score']})")
        checks = pd.DataFrame(risk["checks"])
        st.dataframe(checks)
        st.bar_chart(checks.set_index("check")["score"])
    alerts = [a for a in (call("GET", "/alerts") or []) if a["transaction_id"] == tid]
    if alerts:
        st.write("Alerts sent")
        st.dataframe(pd.DataFrame(alerts))
```

Run it with **streamlit run app.py** and set the backend address with the API_BASE environment variable. A Refresh button in the sidebar reloads the data. Note that a "duplicates" counter needs the backend to count repeated keys, because a duplicate request returns the existing payment and creates no new row; ask Member 1 if you want it on the overview.

What was tested on the reference code: all six pages loaded headlessly against the mock with no errors. The overview counts matched the payments created (4 total, 2 completed, 1 failed, 1 needing reconciliation, 2 review or blocked), and the reconciliation page showed the diagnosis reason.

Step 4. Alerts

Who	Where they see it	Message idea
Sender	Banner in the UPI app; email	Money was deducted but not delivered. Do not retry. Estimated time.
Receiver	Alerts page; email	Payment pending. Do not hand over goods or service yet.
Operations	Alerts page and queue; email	Case opened with reason, severity and action.

The alert rows and their wording come from Members 1 and 2. You make them visible. Do the dashboard and app alerts first. Email is the least important part, so do it last. The reference mailer sends nothing unless SMTP is configured, and only prints what it would send (dry run), and it never sends the same alert twice:

```
def deliver(msg):
    host = os.environ.get("SMTP_HOST")
    if not host:
        print(f"[dry run] to {msg['To']}: {msg['Subject']} | {msg.get_content().strip()}")
        return
    with smtplib.SMTP(host, int(os.environ.get("SMTP_PORT", "587"))) as s:
        s.starttls()
        s.login(os.environ["SMTP_USER"], os.environ["SMTP_PASSWORD"])
        s.send_message(msg)


def run_once(seen):
    for a in requests.get(f"{API}/alerts", timeout=5).json():
        key = (a["transaction_id"], a["recipient"], a["at"])
        if key not in seen:
            deliver(build(a))
            seen.add(key)

if __name__ == "__main__":
```

```
    seen = set()
    while True:
        run_once(seen)
        time.sleep(5)
```

Step 5. End-to-end testing (your main job in Phase 4)

The tests call the API exactly as the screens do. They run against the mock now and against the real backend later by setting environment variables. This is how you find out whether the three tracks truly fit together.

```
API = os.environ.get("API_BASE", "http://localhost:8000")
SENDER = os.environ.get("SENDER", "acc-alice")
RECEIVER = os.environ.get("RECEIVER", "acc-bob")
POOR_SENDER = os.environ.get("POOR_SENDER", "acc-chai")      # an account with a small balance
# An amount that must trigger BLOCK. Agree the trigger with Members 1 and 2.
RISKY_AMOUNT = float(os.environ.get("RISKY_AMOUNT", "60000"))


def set_sim(sender="SUCCESS", receiver="SUCCESS"):
    body = {"sender": sender, "receiver": receiver}
    requests.post(f"{API}/sim/config", json=body).raise_for_status()


def pay(amount=1000, key=None, sender=SENDER, receiver=RECEIVER):
    body = {"idempotency_key": key or str(uuid.uuid4()), "sender_account_id": sender,
            "receiver_account_id": receiver, "amount": amount}
    return requests.post(f"{API}/transactions", json=body)


def timeline_states(tid):
    return [e["to"] for e in requests.get(f"{API}/payment-status/{tid}").json()["timeline"]]


@pytest.fixture(autouse=True)
def reset_sim():
    set_sim()
    yield
    set_sim()


def test_01_normal_payment():
    r = pay().json()
    assert r["state"] == "COMPLETED" and r["duplicate"] is False

def test_08_duplicate_request():
    key = str(uuid.uuid4())
    first, second = pay(key=key).json(), pay(key=key).json()
    assert second["duplicate"] is True
    assert first["transaction_id"] == second["transaction_id"]


def test_09_concurrent_duplicates():
    key = str(uuid.uuid4())
    with ThreadPoolExecutor(20) as pool:
        results = [f.result().json() for f in [pool.submit(pay, 100, key) for _ in range(20)]]
    assert len({r["transaction_id"] for r in results}) == 1
    assert sum(1 for r in results if r["duplicate"] is False) == 1


def test_10_fraud_block():
    r = pay(amount=RISKY_AMOUNT).json()
    assert r["decision"] == "BLOCK" and r["state"] == "FAILED"
    states = timeline_states(r["transaction_id"])
    assert "PROCESSING" not in states, "a blocked payment must not be debited"
```

#	Scenario	How the test triggers it	Expected
1	Normal payment	both banks SUCCESS	COMPLETED, duplicate false
2	Debit ok, credit fails	receiver ERROR	RECONCILIATION_REQUIRED; alerts to sender and ops

#	Scenario	How the test triggers it	Expected
3	Receiver times out	receiver TIMEOUT	RECONCILIATION_REQUIRED with a recovery estimate
4	Credit applied, confirmation lost	receiver LOST_ACK	COMPLETED
5	Debit fails	sender ERROR	FAILED
6	Invalid receiver	unknown receiver account	HTTP 422
7	Insufficient balance	small account sends more than it has	FAILED
8	Duplicate request	same key twice	same payment returned, duplicate true
9	Concurrent duplicates	same key from 20 threads	exactly one payment created
10	Fraud block	amount above the agreed risky value	decision BLOCK, FAILED, never debited

On the mock, all 10 scenarios pass. That only proves the tests and screens are right. The real result comes when you run them against Member 1 and Member 2's code. For every failure, open a GitHub issue against the owning track using the repository template: say which scenario, what you expected and what happened, and attach the response. Record each result and a screenshot in **docs/test-scenarios.md**.

Step 6. The live demo (you drive it)

- **Step 1.** Open the UPI app and the operations dashboard side by side.
- **Step 2.** Send Rs. 1,000: it completes, the balance drops, the history shows it.
- **Step 3.** On Demo controls set the receiver bank to TIMEOUT and apply.
- **Step 4.** Send Rs. 1,000 again. The app shows "Payment under review" with the do-not-retry banner and the recovery estimate.
- **Step 5.** On the dashboard, open the Reconciliation queue: the case is there with its reason, severity and timeline.
- **Step 6.** Open Alerts: sender, receiver and operations alerts were created.
- **Step 7.** Retry the same payment using the same key (use the network-failure test or the API): the existing payment comes back and nothing is debited twice.
- **Step 8.** Set the receiver back to SUCCESS. Send a very large payment to show a fraud block with its per-check reasons.

Rehearse it until it works three times in a row. Make sure the seed data and the simulator switch let you repeat it.

Step 7. Polish

[ ] Every screen has a loading state and a clear error if the backend is down.

[ ] Messages use plain language. No raw state names or error codes in front of the customer.

[ ] Take clean screenshots of each screen and each scenario for the report.

[ ] Check the app on a phone-sized window; it is meant to feel like a mobile payment app.

6. What You Give and Receive

With	You give	You receive
Member 1	The list of extra endpoints and fields you need; issues when the contract is broken	Endpoints, seed data, CORS, simulator switch
Member 2	Questions about field names and wording; issues when output does not match	Per-check scores and reasons, diagnosis reason, severity, action, user message, recovery message
Guide	Test results, screenshots, demo	Sign-off at each checkpoint

Start with the mock backend and publish your list of needed fields on day one. Members 1 and 2 cannot give you what you have not asked for.

7. What You Must Pass

#	Check	Expected
1	UPI app: normal payment	Completes; balance and history update
2	UPI app: empty or zero amount	Clear message; nothing sent
3	UPI app: double tap on Pay	Only one request sent
4	UPI app: stuck payment	Banner with do-not-retry and recovery estimate
5	UPI app: server unreachable	Safe-to-retry message; same key kept
6	Dashboard: overview counts	Match the payments created
7	Dashboard: reconciliation queue	Lists stuck payments; open one for reason, severity, timeline
8	Dashboard: fraud case	Per-check scores and reasons shown
9	Dashboard: backend down	Clear error, no crash
10	All 10 end-to-end scenarios	Pass on the real backend, with results recorded

8. Rules to Follow

Do

- Show reasons, not just labels.
- Keep every API field you use written in the contract.
- Test against the real backend as soon as it exists.
- File an issue for every failure, with the response attached.
- Open a pull request for every change and ask Member 1 or 2 to review it.

Do not

- Decide what a payment state means on screen. The backend decides.
- Hardcode payment logic, thresholds or messages that belong to Members 1 and 2.
- Generate a new idempotency key on a retry after a network error.

- Edit another member's folder. Open an issue instead.
- Claim it is real UPI. It is a simulation.

9. Definition of Done

Phase 0

[ ] Extra endpoints and fields requested from Members 1 and 2 and written in the contract.

Phase 1

[ ] Both screens work on the mock backend.

[ ] Logic tests pass.

Phase 2

[ ] Both screens use the real backend.

[ ] A payment from the app reaches the dashboard.

Phase 3

[ ] Stuck payments are explained in the app and on the dashboard.

[ ] Demo controls switch the simulated banks.

[ ] Alerts appear for sender, receiver and operations.

Phase 4

[ ] All 10 scenarios pass on the real backend.

[ ] Results and screenshots are in docs/test-scenarios.md.

[ ] No open issues against the contract.

Phase 5

[ ] The demo works three times in a row.

[ ] Screenshots are ready for the report.

10. Questions You Should Be Able to Answer

Question	Answer
What does the customer see when a payment is stuck?	A banner saying the money was deducted but not delivered, a clear do-not-retry message, and an estimated resolution time that is shown as an estimate.
Why does your app keep the same key after a network error?	The request may have reached the server. Reusing the key means the retry cannot create a second debit.
Why is the Pay button disabled while waiting?	To stop a double tap sending two requests. The idempotency key is the real protection, the button is a convenience.
Who decides what a state means?	The backend. The screens only display it, so the logic is in one place.
What does the operations dashboard add?	A queue of problem payments, each with its reason, severity, timeline, fraud breakdown and alerts, so an operator can act.
How did you test the whole system?	Ten end-to-end scenarios through the API, triggered by switching the simulated banks, with results and screenshots recorded.
Why a mock backend?	So screens and tests could be built and verified before the real backend was ready.
Is this real UPI?	No. Banks and gateway are simulated to demonstrate the concepts.

11. Common Mistakes to Avoid

- Building screens for fields that were never agreed.
- Showing raw state names or error codes to the customer.
- Using innerHTML for text that comes from the server.
- Testing only against the mock and discovering the mismatch the week before the demo.
- A demo that depends on luck. Use the Demo controls so every failure is repeatable.
- Waiting for the others. Build on the mock and report gaps early.